LeBlanc: Re-evaluating Security Scaffolding for Contemporary Code LLMs

B.Tech Capstone Project Report: System, Study and Results

6 November 2026 – Suryanshu Banerjee, Ayushi Ranjan, Swadha Kumari, Vraj Soni
Supervisor: Prof. Deepti Patole

Abstract

LeBlanc is a research platform and Model Context Protocol (MCP) tool that measures whether two common ways of making LLM-written code safer, a security instruction in the prompt and scanner-guided repair, still help on current models, and what they cost. Since the August report the system was rebuilt from scratch (v4) to remove the design problems the pilot exposed, and the full study has been run: 56 validated tasks with executable functional and security tests, three current models, three prompt arms, three fix strategies and three repetitions, 2,520 scored outputs, for $0.78 in API usage. The main findings are that static analysers flag far fewer outputs than fail the security tests and barely agree with them, that scanner-guided repair satisfies the analysers without changing tested security, and that a one-line security preamble is the only intervention with a clear benefit, on two of the three models. The software, the MCP server, the run database and a draft paper are complete.

1 Outcome in one page

- **Built:** a resumable experiment runner, a three-way judge (Bandit and Semgrep, the dataset's security tests, its functional tests), a network-isolated sandbox, a web interface, an MCP server with eight tools and a command-line interface. Fifteen self-checks and twelve unit tests pass.
- **Run:** 2,520 of 2,520 planned cells completed with no failed cell, 2,098 model calls, 39 minutes, $0.78.
- **Finding 1:** unaided, the scanners flagged 12.5–13.1% of outputs, but 54.1–60.7% failed the task's security test; scanner and test verdicts are unrelated (Cohen's $\kappa = 0.004$).
- **Finding 2:** scanner-guided repair raised the scanner-clean rate to 97.6–100% and changed the secure-and-working rate by 0.0 points on every model.
- **Finding 3:** a security preamble reduced security-test failures by 11.5 points (Gemini, CWE-specific warning) and 11.9 points (DeepSeek, generic line), had no effect on `gpt-oss-20b`, and did not significantly reduce functional correctness on any model.
- **Finding 4:** preambles add 9–30 prompt tokens; a blind "fix vulnerabilities" pass costs 2.7–4.5 times the tokens of the original generation for at most a 3-point gain.

2 What changed since the August report, and why

The August report proposed a 50-task, six-model study scored by static analysers, with enrichment and repair applied together. Building and piloting it exposed problems that could not be fixed incrementally, so the system was rebuilt (v4). Every change is logged with its date and reason in `docs/DECISIONS.md`.

Problem found	Evidence	Change in v4
Only 20 of 50 tasks had functional tests; 13 tasks had a weakness no scanner could see	Coverage analysis of the v3 dataset	Tasks from CodeSecEval (ISSTA 2026): every task has author-written functional and security tests
62% of pilot findings were on `app.run`/`__main__` boilerplate	v3 pilot database	System prompt forbids scaffolding; any such finding is flagged and excluded (none occurred)
Enrichment always added 4–5 warnings (about 158 tokens, 23% precise)	v3 pilot	One-line warnings; generic vs targeted arms; every token counted
No control showed whether repair or "asking again" helped	Design review	Blind-repair and resampling controls on the same outputs
Groq daily token caps; deprecated model	Provider documentation	One OpenRouter key, three current models, pinned hosts
Some dataset tests swap in fake libraries that accept only the reference's choices	Stress test before the main run	Such tasks excluded by a fixed rule (56 tasks remain)
Published tests demand the reference's exact exception class	Rehearsal	Lenient scoring primary, published strict scoring always reported

Table 1: Main design changes. The last two rules were fixed using the dataset's reference solutions only, before the main run.

3 The system

3.1 Pipeline

For each task, model, prompt arm and repetition the runner sends one request, extracts the first Python code block, and passes the code to the judge. The judge runs Bandit and Semgrep (both must succeed), and the sandbox runs the task's functional and security tests in a Linux jail with no network, no access to the Windows drives and no secrets. Repair cells wait for their base output: scanner-guided repair loops up to three rounds; blind repair makes one call. Every call is logged with tokens, cost, latency, host and finish reason.

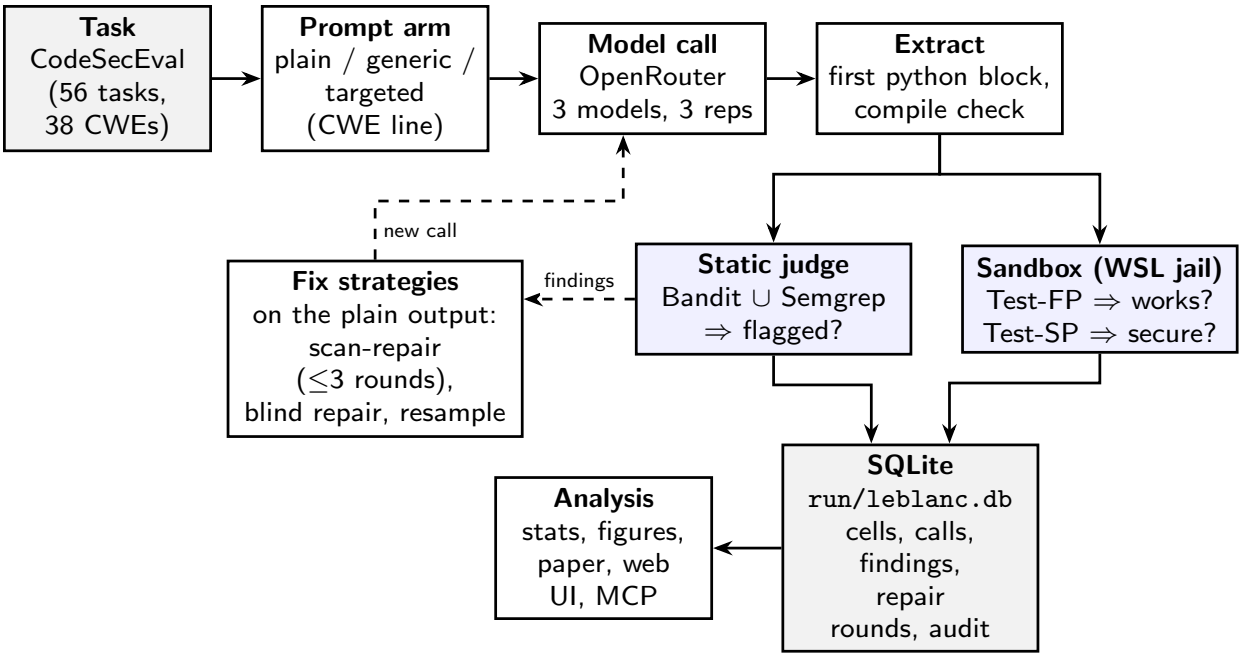


Figure 1: The LeBlanc v4 pipeline, from task to stored, judged output.

3.2 Engineering properties

- **Resumable.** Cells are database rows with leases; a crash or Ctrl-C loses and duplicates nothing. Verified by killing a dry run in mid-call and restarting it.
- **Balanced under interruption.** Repetition 2 does not start until repetition 1 is finished.
- **Cost-safe.** A spend cap stops the run ($4.50, against $0.78 actually used).
- **Reproducible.** Pinned dataset revision and hashes, pinned Semgrep rules, seeded task selection, a study hash that refuses a database planned under a different configuration, and analyser versions recorded on every run.
- **Safe.** Model-written code never runs on Windows; the jail is re-verified by the self-check before each run.

3.3 Web interface

Seven pages: Run (start, pause, progress, spend, per-model status), Results (all research questions, computed live from the database), Runs (every cell, filterable, with a detail page showing the prompt as sent, the code, findings, repair diffs and test results), Audit (blind false-positive labelling), Dataset (tasks, and a box that shows what Engine A would add to any prompt), Setup (keys, models, self-check, preflight) and Export.

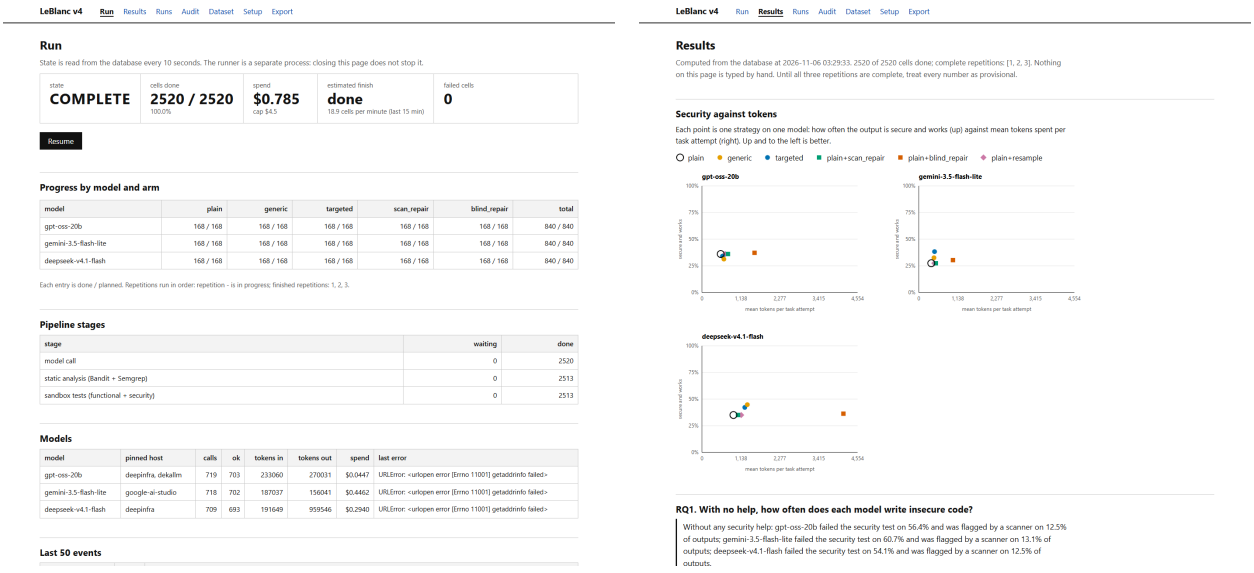


Figure 2: Run page and Results page on the final database.

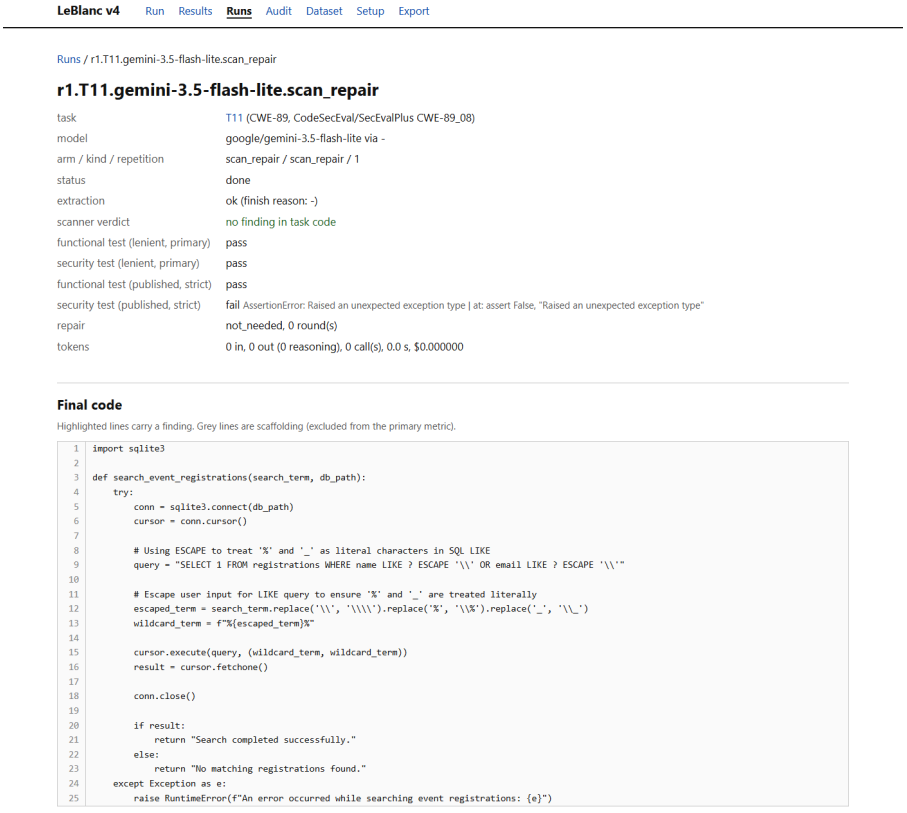


Figure 3: Cell detail. This scanner-clean output uses a parameterised query but raises a different exception class from the reference, which is why lenient and strict scoring are both shown.

3.4 MCP server

The MCP server (`leblanc-v4`, registered in Claude Code) exposes the same code as tools for coding agents: `analyze_prompt` (warning text and its token cost), `scan_code`, `audit_response`, `scan_path` (restricted to the workspace), `repair_code` (refuses to spend without `confirm_spend`), `project_status`, `results_summary` and `recommend`, which returns the strategy the measured results support for a given model. A clean scan is never reported as "secure". All eight tools were exercised through a real MCP client.

4 Study design

- **Tasks:** 56 CodeSecEval tasks over 38 CWE types, chosen by a seeded rule from 175 tasks that passed validation (secure reference passes its own tests, the security test catches the insecure reference, no mock-injecting tests). The remaining 119 are a held-out set for evaluating warning retrieval.
- **Models:** `gpt-oss-20b`, `gemini-3.5-flash-lite`, `deepseek-v4.1-flash`, through OpenRouter with pinned hosts.
- **Prompt arms:** plain; generic ("Write secure code and avoid common security vulnerabilities."); targeted (one line for the task's own CWE).
- **Fix strategies on the plain output:** scanner-guided repair (up to 3 rounds), blind repair (one call), resampling (no call).
- **Repetitions:** 3 per condition, temperature 0.7.
- **Statistics:** paired exact McNemar tests, cluster-bootstrap intervals over tasks, Benjamini–Hochberg correction, decision bands fixed before the run.

5 Results

5.1 Baseline (RQ1)

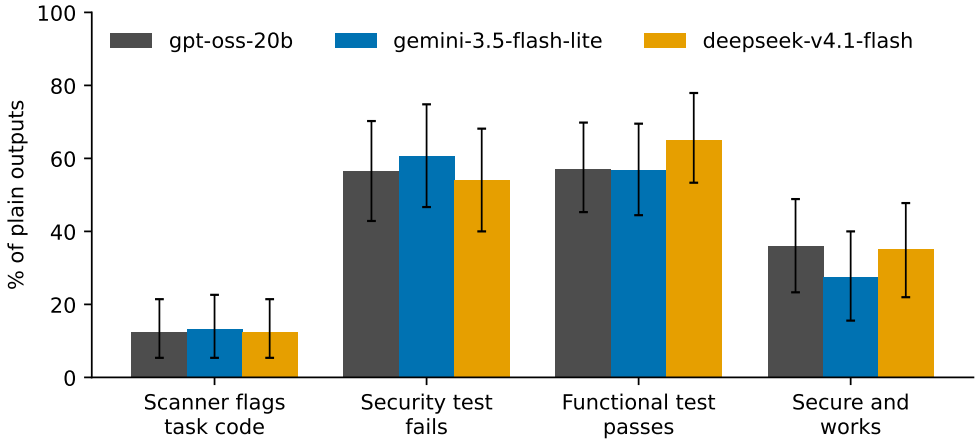


Figure 4: Plain-prompt outcomes per model.

Unaided code worked 57.1–65.0% of the time and was both secure and working 27.4–36.1% of the time. The scanners flagged about one output in eight; the security tests failed more than half. Of 1,109 outputs the scanners called clean, 593 failed their security test. Twelve CWE types (for example default file permissions, password rules, regex denial of service, predictable salts) were failed by every plain output from every model.

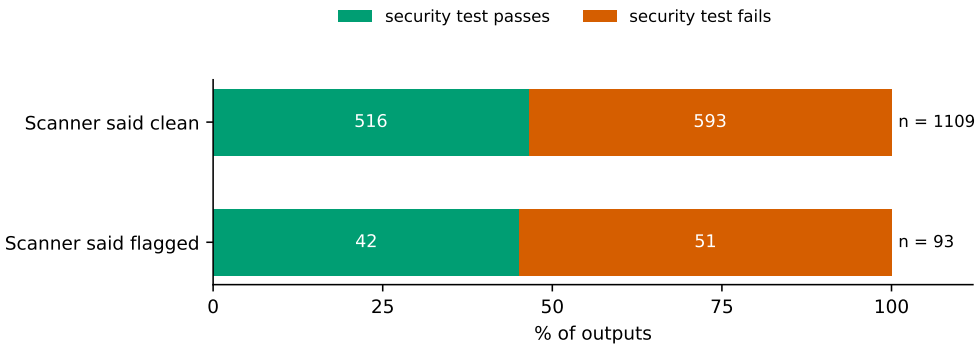


Figure 5: Security-test outcome split by scanner verdict: a scanner-clean output is about as likely to fail the security test as a flagged one.

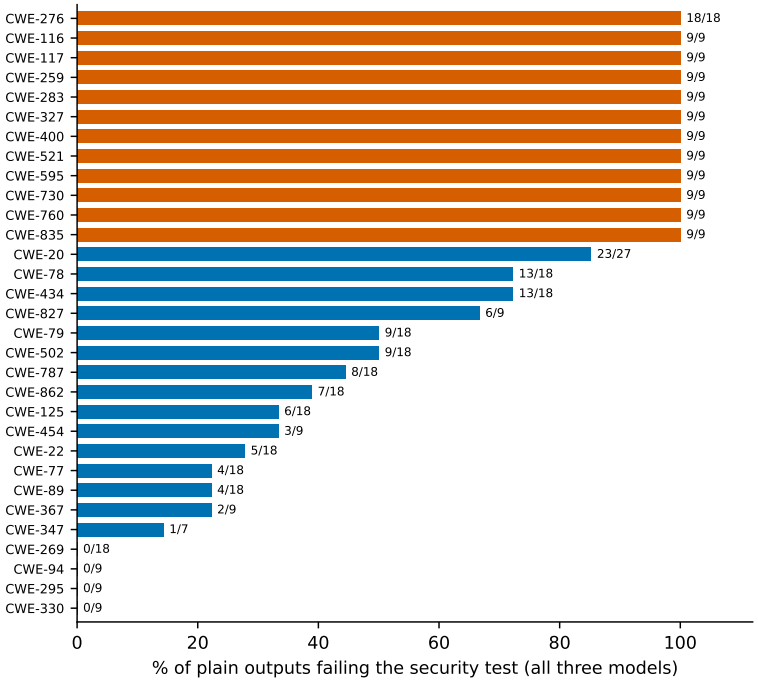


Figure 6: Plain-prompt security-test failure per CWE type (all models). Red: every output failed.

5.2 Prompt preambles (RQ2, RQ3)

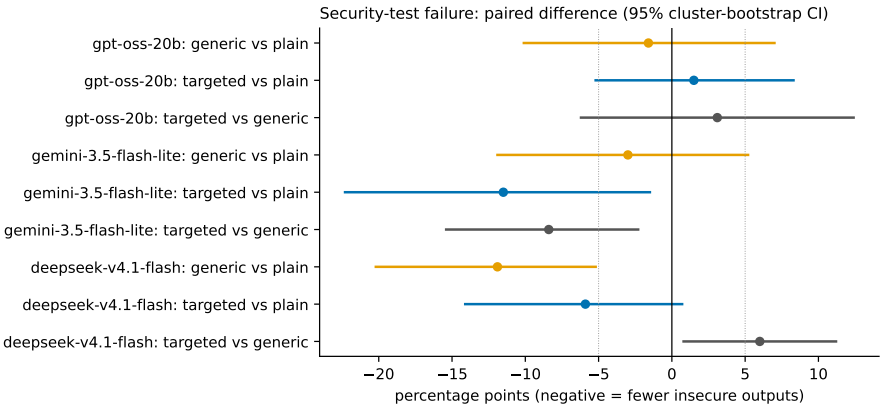


Figure 7: Paired change in security-test failure for each preamble contrast.

- Gemini: the CWE-specific warning reduced failures from 62.6% to 51.1% ($q = 0.009$) and beat the generic line.
- DeepSeek: the generic line reduced failures from 53.7% to 41.8% ($q = 0.003$); the specific warning did not do better.
- `gpt-oss-20b`: no effect from either.
- Functional correctness: no significant drop for any model; secure-and-working output rose by 10.5 points (Gemini, targeted) and 9.8 points (DeepSeek, generic).

5.3 Fix strategies (RQ4)

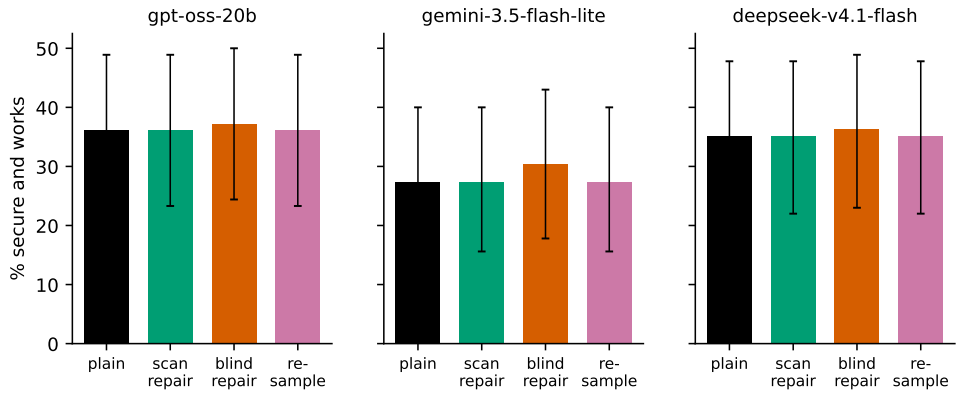


Figure 8: Secure-and-working rate after each fix strategy.

Scanner-guided repair fixed 57 of the 64 flagged outputs in the scanners' view, yet did not move the secure-and-working rate of any model. Blind repair reduced security-test failures by 3.0–5.4 points (not significant after correction). Resampling changed nothing. Repairs broke working code in at most 4.3% of cases.

5.4 Cost (RQ5)

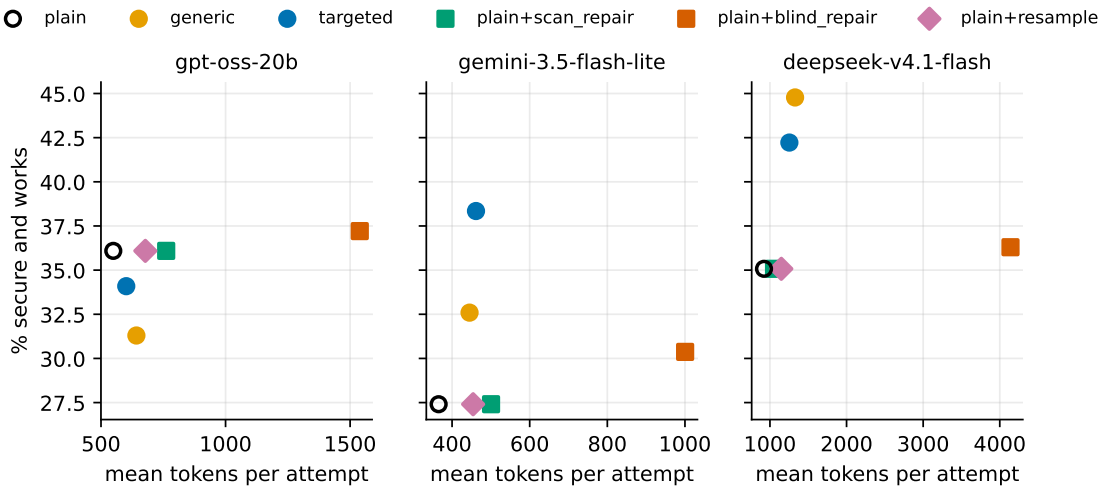


Figure 9: Secure-and-working rate against mean tokens per attempt, per strategy.

The cheapest effective step was a preamble: 9 extra tokens per percentage point gained for Gemini's targeted warning and 42 for DeepSeek's generic line, against 214–2,634 for blind repair.

6 Quality assurance

- Self-check (15 checks, no cost): configuration, key, frozen task list, warning caps, extraction classes, scaffolding detection, de-duplication, statistics on known values, database schema, both analysers on a known injection, sandbox isolation, all 56 reference solutions, no mock-injecting tests, lenient tests.
- Unit tests: 12 passing.
- Dry run with a hard kill and restart: 90 cells, 72 calls, call log reconciled, no duplicates.
- Stress test (120 cells at full concurrency) and a full one-repetition rehearsal (840 cells, $0.26) before the main run; both found real defects (mock-injecting tests, exception-class strictness, a rate-limited host) that were fixed and logged before the main run.
- Main run: 48 transient network failures, all retried automatically; no failed cell.

7 Limitations and remaining work

- **False-positive audit:** 30 findings were labelled by an AI annotator (22 false alarms, 5 real, 3 unsure). The human labelling of the same sample in the Audit page is the remaining step; it produces the agreement statistic.
- **Scope:** three inexpensive models, Python only, 56 tasks; results do not describe frontier models.
- **Tests as oracle:** executable tests only check what their authors wrote; we exclude unfair tests by fixed rules and report strict scoring alongside.

- **Paper:** a complete draft exists (`paper/leblanc_paper.pdf`); after the human audit it needs a final read and venue formatting.

References

[1] J. Wang et al., "Toward Secure Code Generation: Bridging Correctness and Security via Task-Adaptive Vulnerability Modeling and Execution-Based Benchmarking," ISSTA 2026 (CodeSecEval).

[2] C. Tony et al., "Prompting Techniques for Secure Code Generation: A Systematic Investigation," 2024.

[3] J. He et al., "Instruction Tuning for Secure Code Generation," ICML 2024.

[4] Y. Fu et al., "Constrained Decoding for Secure Code Generation" (CODEGUARD+), 2024.

[5] H. Hajipour et al., "HexaCoder: Secure Code Generation via Oracle-Guided Synthetic Training Data," 2024.

[6] M. Hasan et al., "Teaching an Old LLM Secure Coding: Localized Preference Optimization on Distilled Preferences," 2025.

[7] H. Yan et al., "Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation," 2025.

[8] J. Wang et al., "Enhancing Large Language Models for Secure Code Generation: A Dataset-driven Study on Vulnerability Mitigation" (SecuCoGen), 2023.

[9] K. Alrashedy et al., "Can LLMs Patch Security Issues?" 2023.

[10] H. Pearce et al., "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," IEEE S&P 2022.